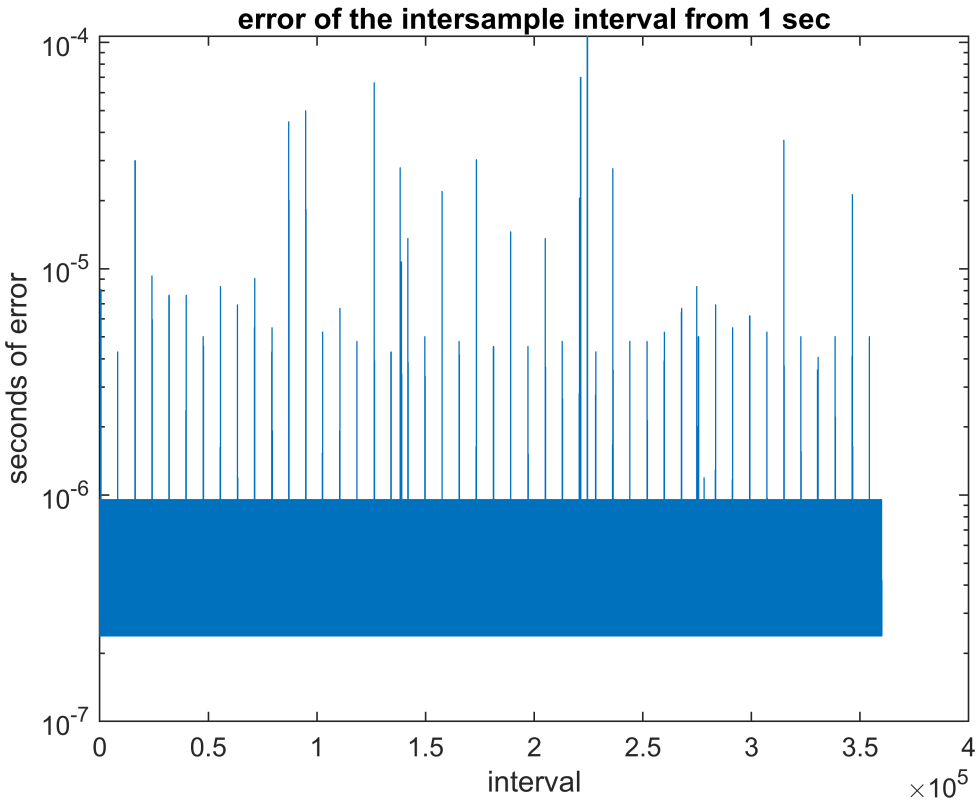
```
clear
clc
%profile on

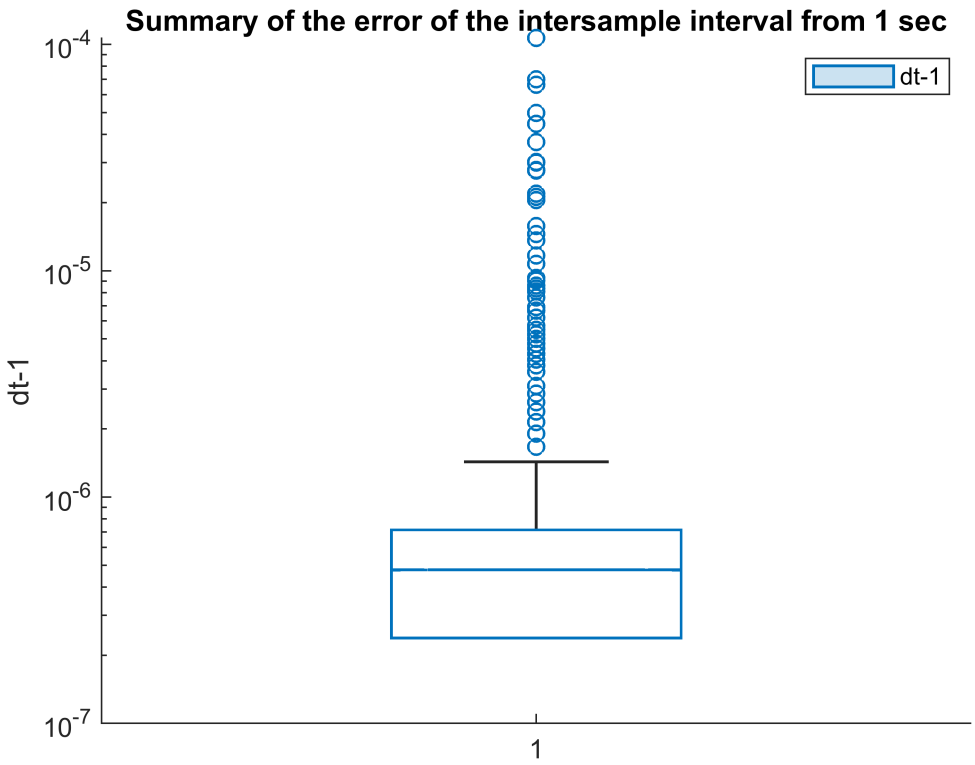
cfg = config.getConfig();

%% Load
TimeTemp = io.loadTimeTemp();

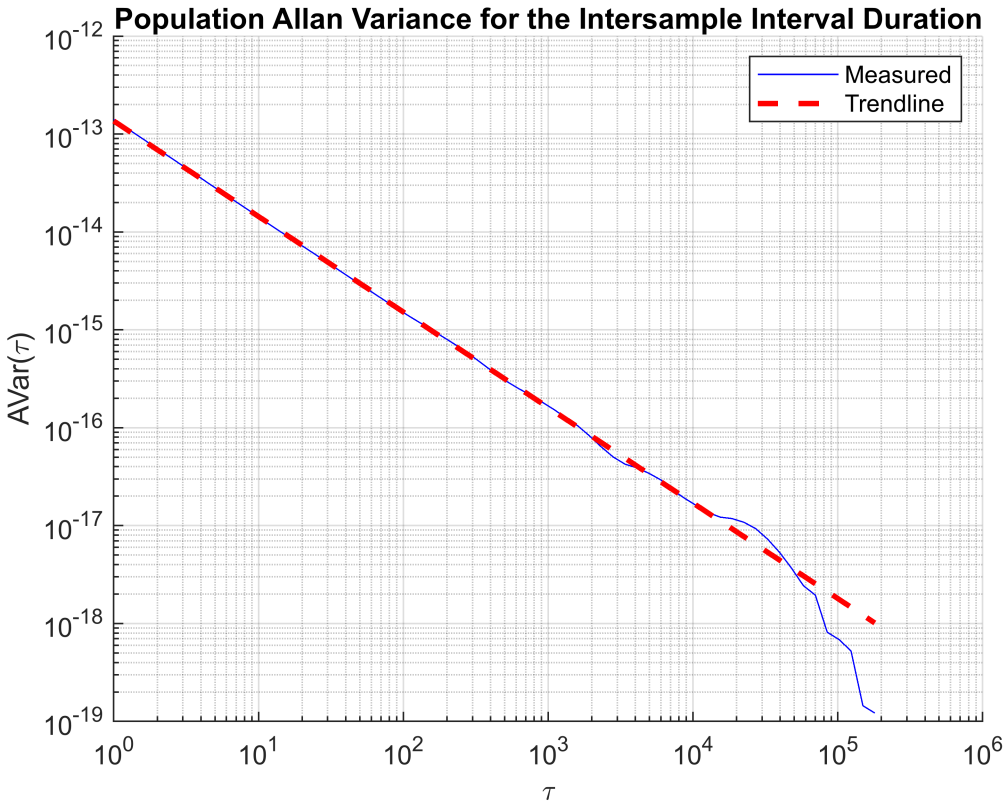
%% Timing analysis
[dt,dt_stats] = preprocessing.computeTimingJitter(TimeTemp);
plot.plot_dt_IntersampleInterval(dt) %figure
```




```
plot.plot_dt_boxchart(dt) %figure 1
```

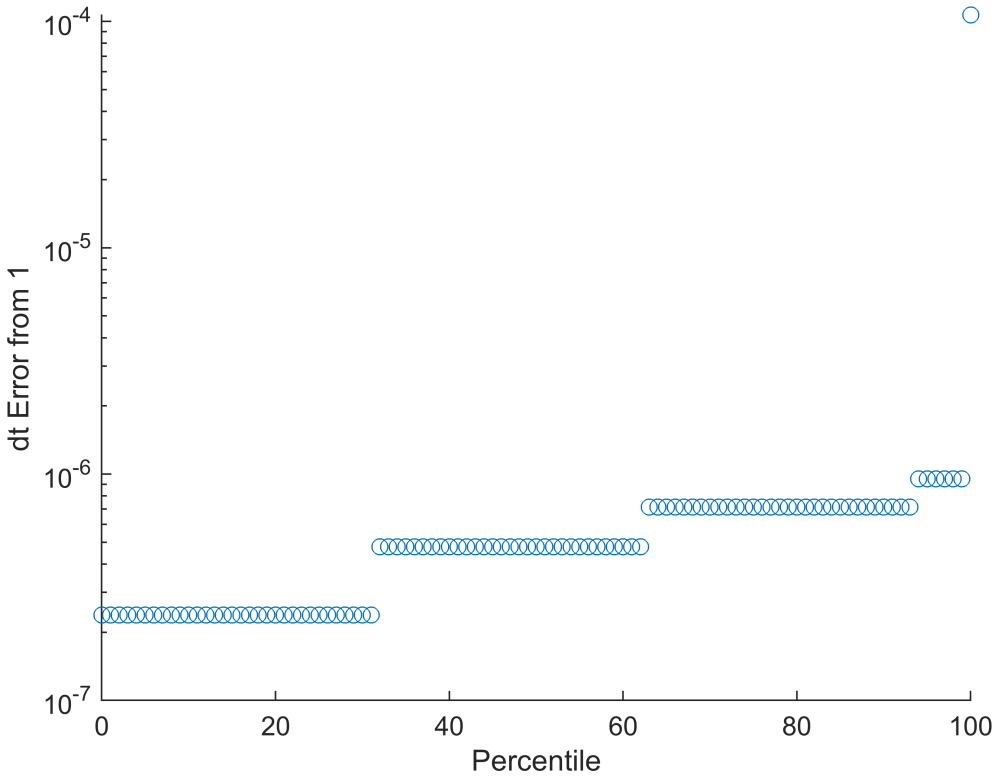



```
plot.plot_dt_avar(dt_stats) %Figure 2
```



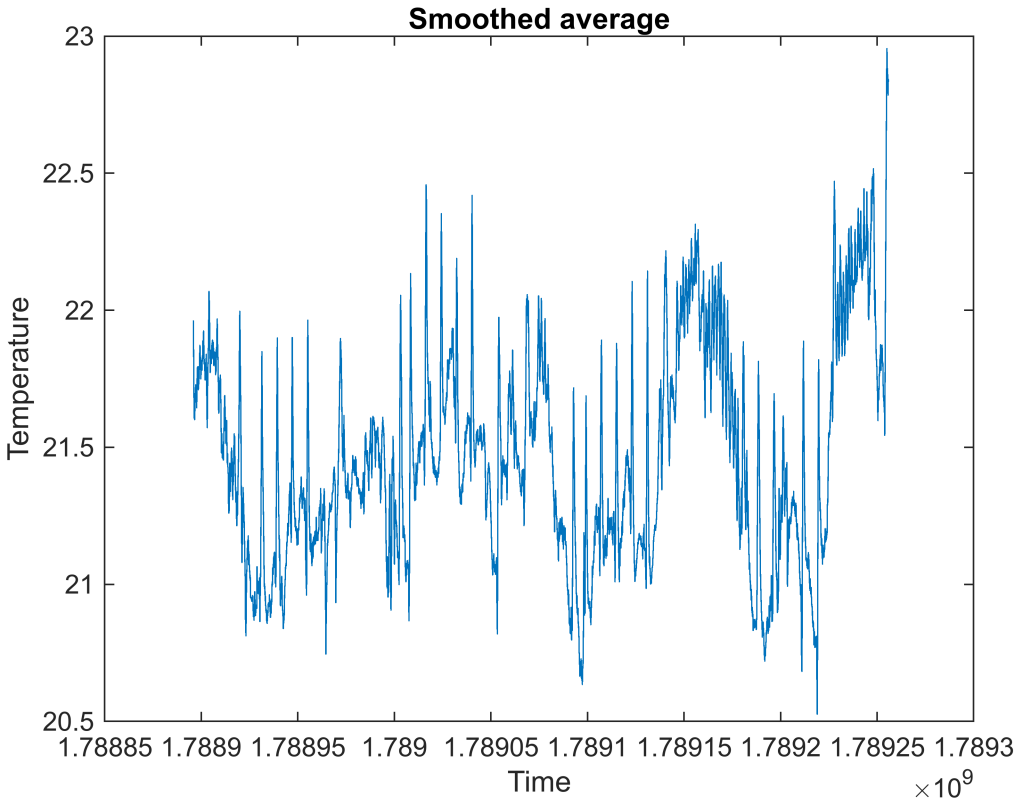
```
LogLog Slope:-0.97566
LogLog Intercept:-12.8669
AVAR(\tau)=-12.8669\tau^-0.97566
```

```
plot.plot_dt_percentile(dt_stats) %figure 3
```




```
%% Signal preprocessing
[robustAvgSmoothGauss,robustAvg] = preprocessing.computeRobustAverage( ...
    TimeTemp{:,cfg.sensorNames}, ...
    TimeTemp{:,"now"}, ...
    cfg);

plot(TimeTemp{:,"now"},robustAvgSmoothGauss); %Figure 4
xlabel("Time");ylabel("Temperature");title("Smoothed average")
```




```
noise =
preprocessing.computeNoise(TimeTemp{:,cfg.sensorNames},robustAvgSmoothGauss);

combinedSignal = [robustAvgSmoothGauss-mean(robustAvgSmoothGauss),noise];
signalNames = ["zeroed_rAveSG",cfg.sensorNames];

%% Full-record fingerprint

datafeatures = features.extractFingerprint(combinedSignal,signalNames);

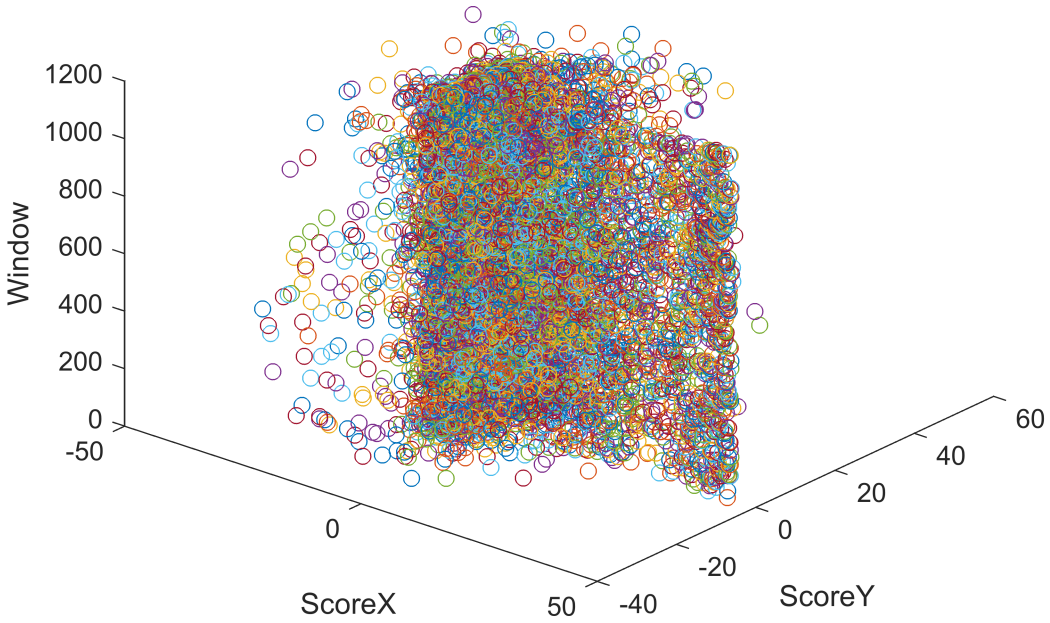
[coeff,score,latent,tsquared,explained,mu] = analysis.runPCA(datafeatures);
%plot.plotPCAscore([score(:,1) score(:,2)],[explained(1) explained(2)]) %figure 5


%Rolling on the data
windowLength = 600;%maxpts = 600; %seconds of window
windowStep = 300;
windowOffsets = 0:windowLength-1;%windowpoints = 0:maxpts-1;
lastStart = size(combinedSignal,1)-windowLength+1;%seconds=height(combinedSignal)-
maxpts;
windowStarts = 1:windowStep:lastStart;%slices = 1:maxpts/2:seconds;
nWindows = numel(windowStarts); %nWindows=length(slices);
Fingerprints = cell(nWindows,1);

for i=1:nWindows
```

```
    t=windowStarts(i);
    f = features.extractFingerprint(combinedSignal(t+windowOffsets,:),signalNames);
    %Fingerprints{i}.Table = f;
    Fingerprints{i}.Matrix = table2array(removevars(f,"SensorName"));
    if i == 1
        rollingFeatureTemplate = f;
    end
end
figure
hold on
for i=1:nWindows
    [coeff,score,latent,tsquared,explained,mu] =
analysis.runPCA(Fingerprints{i}.Matrix);
    XY = score(:,1:2);
    XY = XY - mean(XY,1);% Center layer
    vRef = XY(1,:);% Reference is zeroed_rAveSG, row 1
    theta = atan2(vRef(2),vRef(1));% Angle of reference
    R = [cos(-theta) -sin(-theta);sin(-theta) cos(-theta)];% Rotate reference onto
+X axis
    XYaligned = XY * R';

    assert(XYaligned(1,1) >= -1e-10)
    assert(abs(XYaligned(1,2)) < 1e-10)
    scatter3(XYaligned(:,1), XYaligned(:,2), i*ones(size(XYaligned,1)))
end
xlabel("ScoreX");ylabel("ScoreY");zlabel("Window");
hold off
view(40,35)
```




```
%% Temporal fingerprints

%flatten the vectors for processing
FeatureMatrixRaw = zeros(nWindows,numel(Fingerprints{1}.Matrix));

for i = 1:nWindows
    FeatureMatrixRaw(i,:) = Fingerprints{i}.Matrix(:);
end
size(FeatureMatrixRaw);
%% Build flattened rolling feature names
featureNames =
string(removevars(rollingFeatureTemplate,"SensorName").Properties.VariableNames);
sensorNames = string(rollingFeatureTemplate.SensorName);
flatFeatureNames = strings(numel(sensorNames) * numel(featureNames),1);
idx = 1;

for feature = 1:numel(featureNames)
    for sensor = 1:numel(sensorNames)
        flatFeatureNames(idx) = sensorNames(sensor) + "__" + featureNames(feature);
        idx = idx + 1;
    end
end
assert( ...
    numel(flatFeatureNames) == size(FeatureMatrixRaw,2), ...
```

```
    "Feature names do not match flattened fingerprint width.");
%% Validate temporal feature matrix

featureStd = std(FeatureMatrixRaw,0,1);

constantCols = featureStd == 0;
nonFiniteCols = any(~isfinite(FeatureMatrixRaw),1);

fprintf("Constant feature dimensions: %d\n",nnz(constantCols));
```

```
Constant feature dimensions: 151
```

```
fprintf("Nonfinite feature dimensions: %d\n",nnz(nonFiniteCols));
```

```
Nonfinite feature dimensions: 0
```

```


validCols = ~constantCols & ~nonFiniteCols;

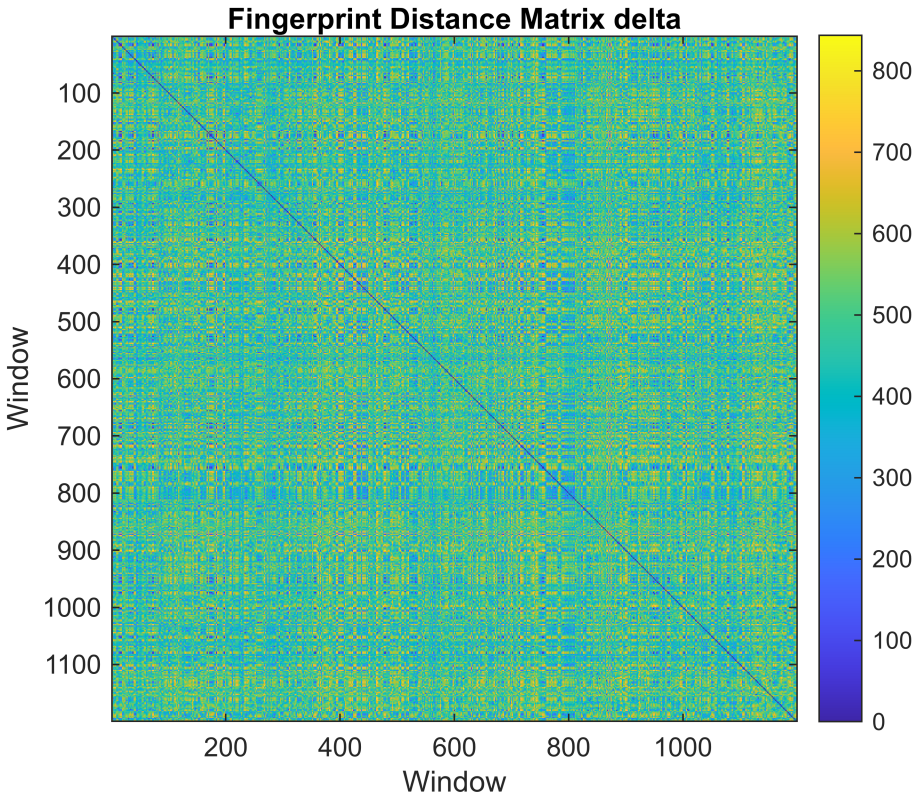
FeatureMatrix = FeatureMatrixRaw(:,validCols);

fprintf("Accepted: %d / %d feature dimensions\n", ...
    width(FeatureMatrix), ...
    width(FeatureMatrixRaw));
```

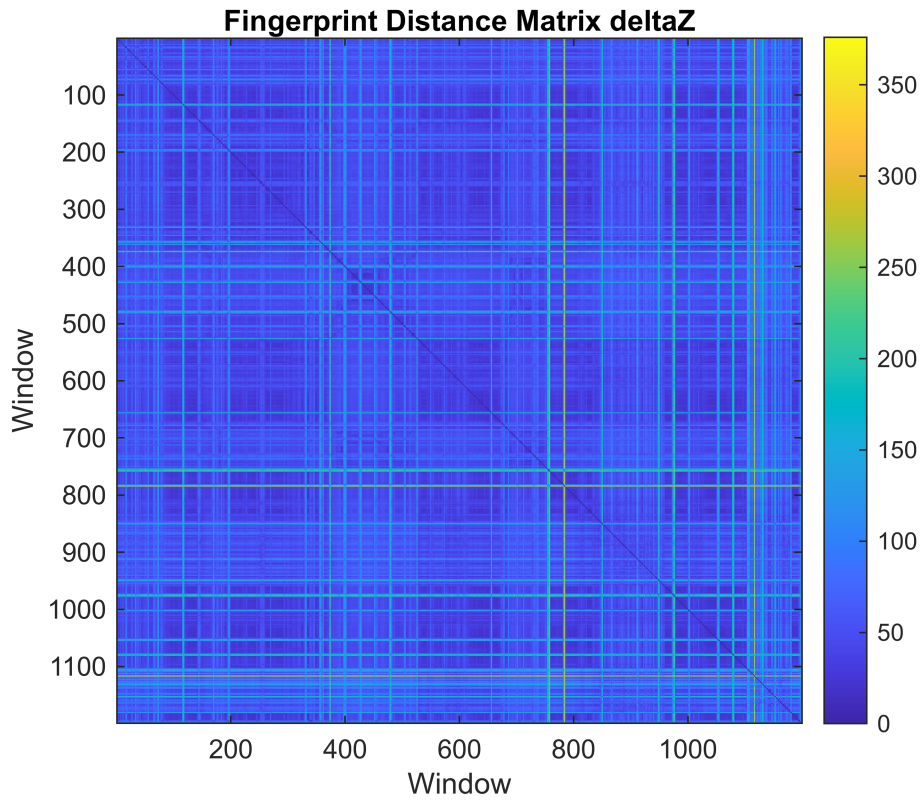
```
Accepted: 2936 / 3087 feature dimensions
```

```
acceptedFeatureNames = flatFeatureNames(validCols);
rejectedFeatureNames = flatFeatureNames(~validCols);
%% Deltas
% deltaRaw: absolute fingerprint difference in the original feature scales
deltaRaw = squareform(pdist(FeatureMatrix,'euclidean'));
% deltaZ: standardized fingerprint difference, where high-magnitude features don't
automatically dominate
deltaZ = squareform(pdist(zscore(FeatureMatrix),'euclidean'));

figure
imagesc(deltaRaw) %Figure 6
axis image
colorbar
xlabel('Window');ylabel('Window');title('Fingerprint Distance Matrix delta')
```



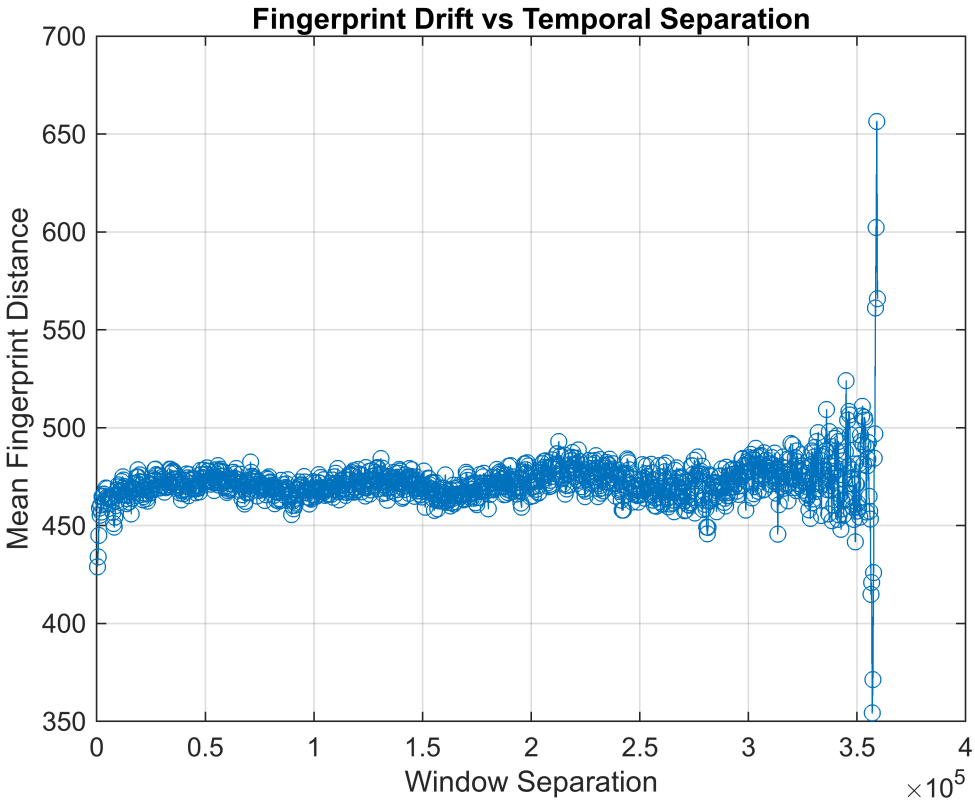

```
figure
imagesc(deltaZ) %figure 7
axis image
colorbar
xlabel('Window');ylabel('Window');title('Fingerprint Distance Matrix deltaZ')
```



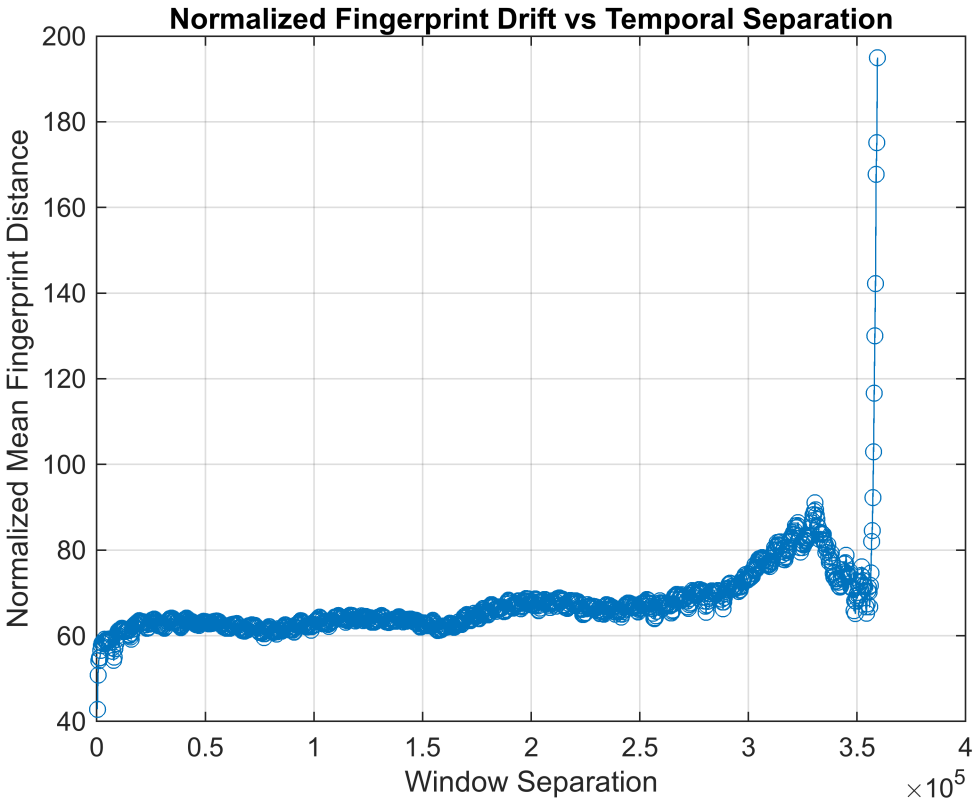

```
% Drift Measurement

nLags = size(deltaRaw,1)-1;
driftRaw = zeros(nLags,1);
driftZ = zeros(nLags,1);
for k = 1:nLags
    driftRaw(k)=mean(diag(deltaRaw,k));
    driftZ(k)=mean(diag(deltaZ,k));
end

lagSamples = (1:nLags) * windowStep;
lagSeconds = lagSamples / cfg.sampleRate;
figure
plot(lagSeconds,driftRaw,'o-')
xlabel("Window Separation")
ylabel("Mean Fingerprint Distance")
title("Fingerprint Drift vs Temporal Separation")
grid on
```




```
figure
plot(lagSeconds,driftZ,'o-')
xlabel("Window Separation")
ylabel("Normalized Mean Fingerprint Distance")
title("Normalized Fingerprint Drift vs Temporal Separation")
grid on
```



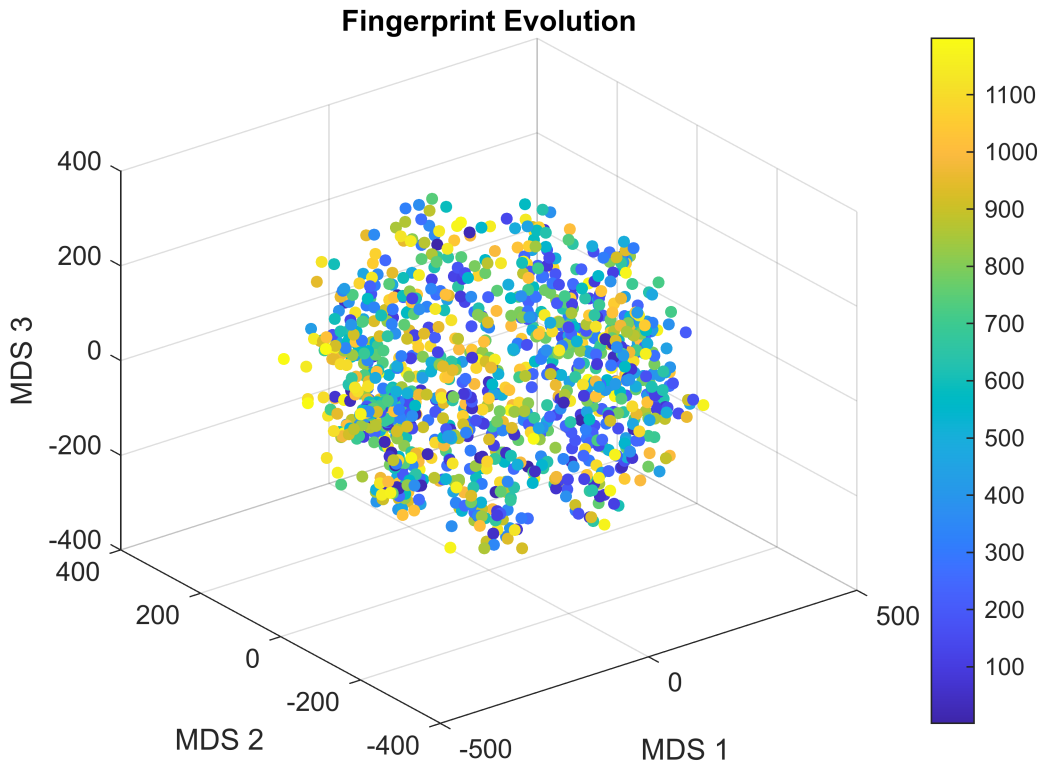

```
%% MultiDimentional Scale
tic
Yraw = mdscale(deltaRaw,3);
toc
```

```
Elapsed time is 27.457106 seconds.
```

```
Yz   = mdscale(deltaZ,3);
toc
```

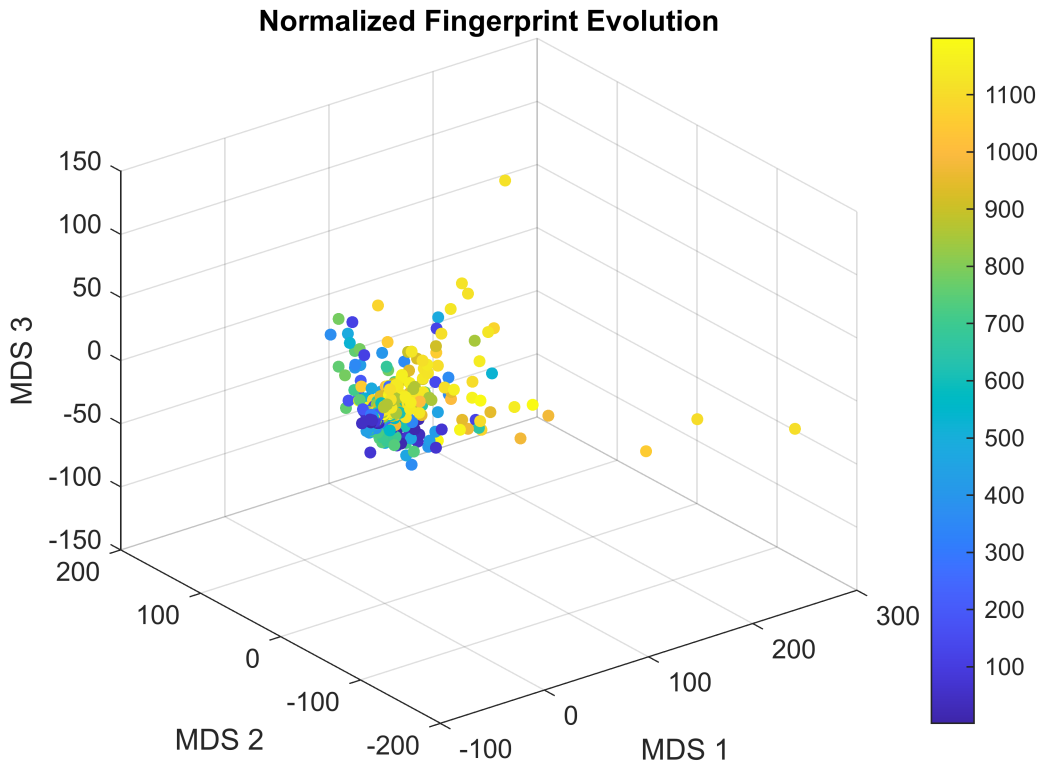
```
Elapsed time is 190.136273 seconds.
```

```
figure
scatter3(Yraw(:,1),Yraw(:,2),Yraw(:,3),20,1:size(Yraw,1),'filled')
xlabel("MDS 1");ylabel("MDS 2");zlabel("MDS 3");
colorbar
title("Fingerprint Evolution")
```



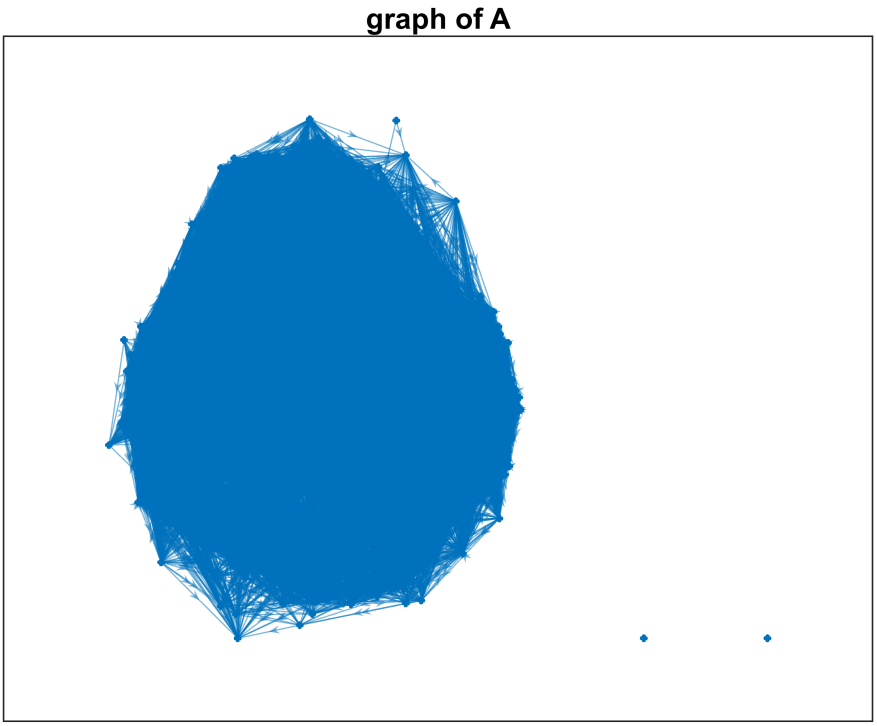

```
figure
scatter3(Yz(:,1),Yz(:,2),Yz(:,3),20,1:size(Yz,1),'filled')
colorbar
xlabel("MDS 1");ylabel("MDS 2");zlabel("MDS 3");

title("Normalized Fingerprint Evolution")
grid on
```

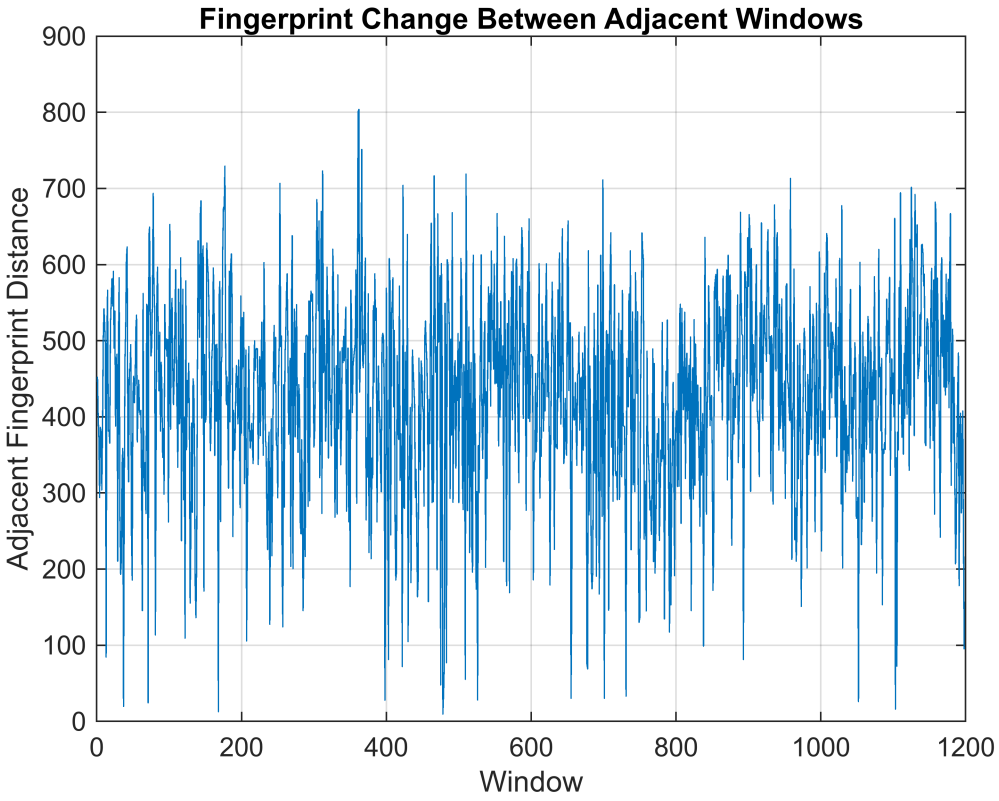



```
%% build a Directed Acyclic Graph
figure %figure 10
upperDistances = deltaRaw(triu(true(size(deltaRaw)),1));
threshold = prctile(upperDistances,10);

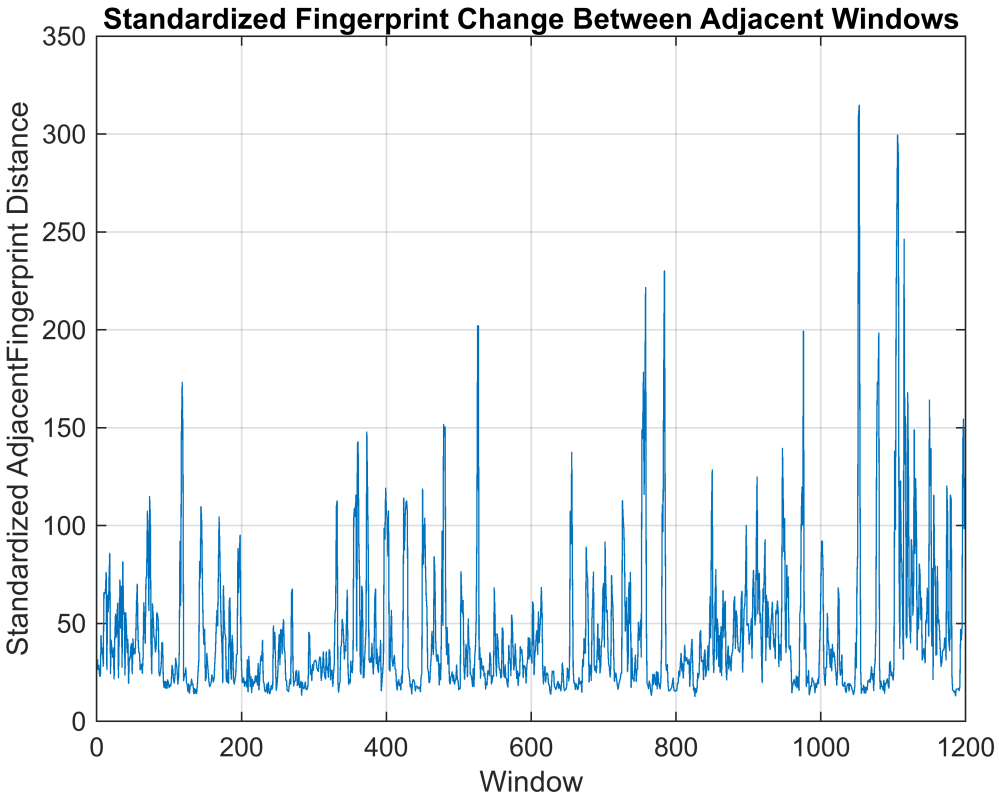
A = deltaRaw < threshold;
A = triu(A,1);
G = digraph(A);
plot(G)
title("graph of A")
```



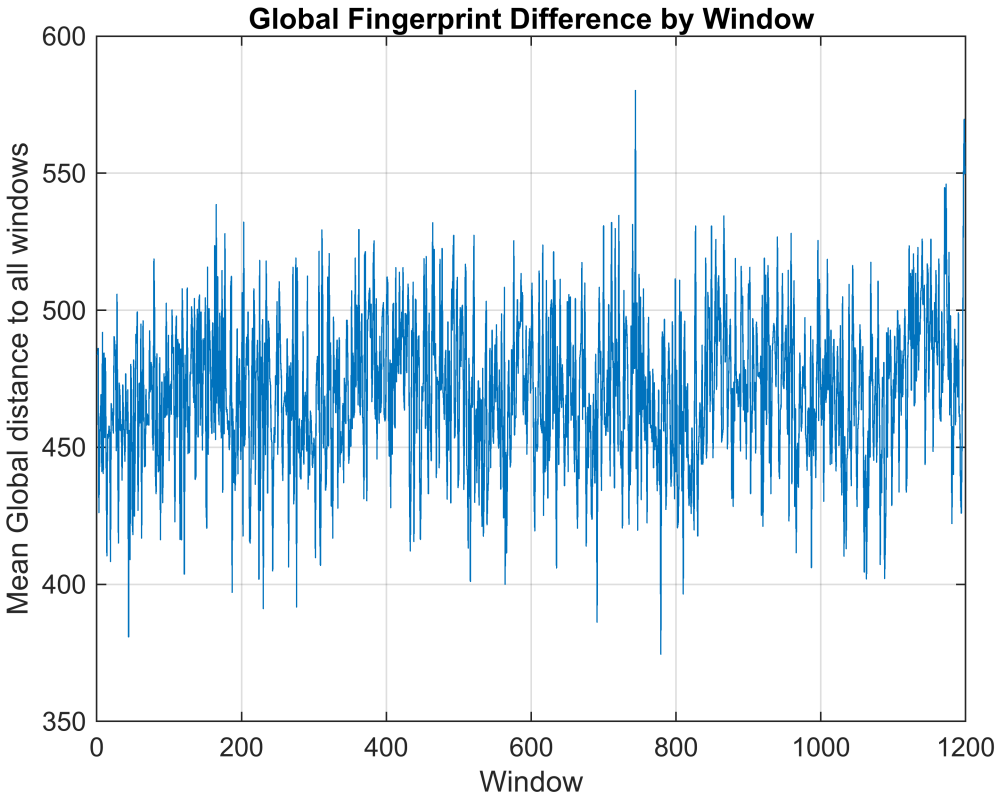

```
%% Detect regime changes
figure
adjacentDistanceRaw = diag(deltaRaw,1);
plot(adjacentDistanceRaw)
xlabel("Window");ylabel("Adjacent Fingerprint Distance");
title("Fingerprint Change Between Adjacent Windows");
grid on
```




```
figure
adjDistanceZ = diag(deltaZ,1);
plot(adjDistanceZ);
xlabel("Window");ylabel("Standardized AdjacentFingerprint Distance");
title("Standardized Fingerprint Change Between Adjacent Windows")
grid on
```



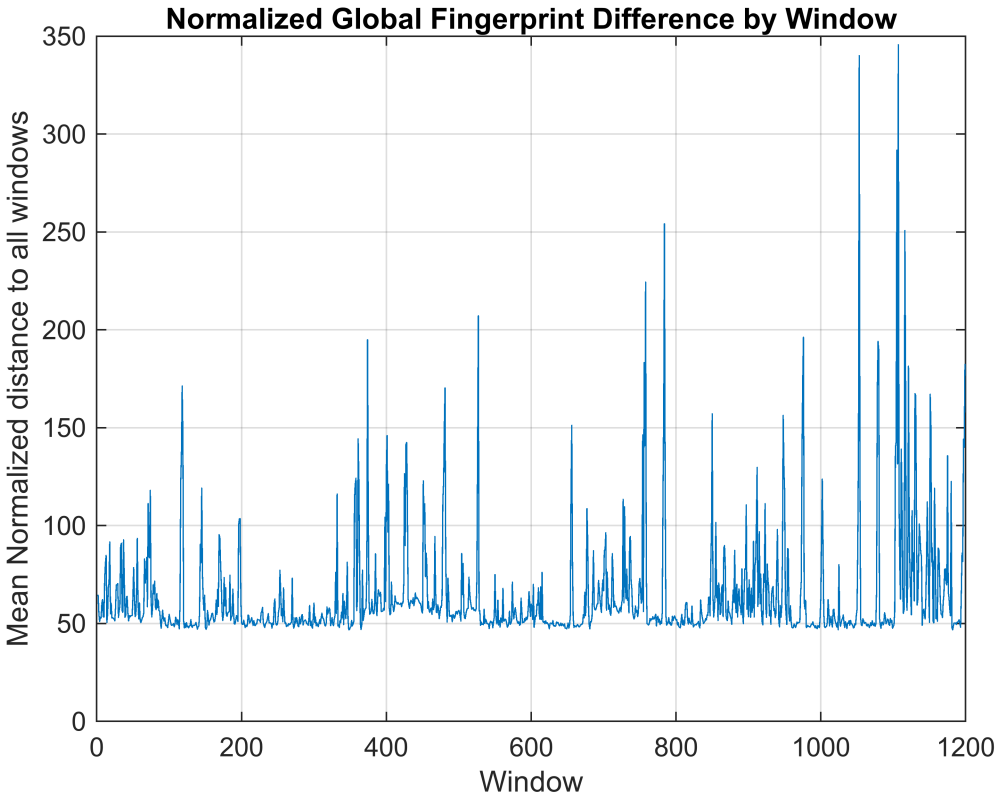

```
figure
meanDistanceRaw = mean(deltaRaw,2);
plot(meanDistanceRaw)
xlabel("Window");
ylabel("Mean Global distance to all windows");
title("Global Fingerprint Difference by Window");
grid on
```




```
figure
meanDistanceZ = mean(deltaZ,2)
```

```
meanDistanceZ = 1199×1
   64.6487
   64.3029
   54.8222
   50.9757
   53.1567
   52.7794
   60.1799
   62.2398
   53.3376
   53.3239
      ⋮
```

```
plot(meanDistanceZ)
xlabel("Window");
ylabel("Mean Normalized distance to all windows");
title("Normalized Global Fingerprint Difference by Window");
grid on
```




```
%profile viewer
```